

Day 7: MySQL 数据库基础 - 学习总结

日期: 2026-06-29

一、MySQL 基础概念

1. 为什么用 MySQL?

特性	CSV 文件	MySQL 数据库
数据量	几千行还行	百万行也不卡
查询速度	每次读整个文件	有索引, 秒查
多人同时用	容易出问题	支持并发
数据类型	全是文本	强制类型 (整数/日期/文本)

2. 核心概念

- **数据库 (Database)** : 一组相关的表, 类比一个项目文件夹
- **表 (Table)** : 类似 pandas 的 DataFrame, 有行有列
- **字段 (Column/Field)** : 列, 比如"姓名""年龄"
- **记录 (Row/Record)** : 一行数据
- **主键 (Primary Key)** : 唯一标识每一行的列 (不能重复)

二、MySQL 安装与连接

1. 验证安装

```
# 在 PowerShell 中  
mysql --version
```

2. 连接 MySQL

```
# 使用完整路径连接（如果环境变量未配置）  
& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u root -p  
  
# 参数说明：  
# -u root: 使用 root 用户登录（管理员账户）  
# -p: 需要输入密码（输入时屏幕不显示字符，这是安全设计）
```

3. 退出 MySQL

```
# 方法1: 输入命令  
EXIT;  
  
# 方法2: 快捷键  
Ctrl+C
```

三、SQL 基本命令

1. 数据库操作

查看所有数据库

```
SHOW DATABASES;
```

说明：显示 MySQL 中所有的数据库列表

创建数据库

```
CREATE DATABASE 数据库名;
```

示例:

```
CREATE DATABASE student_system;
```

使用数据库

```
USE 数据库名;
```

示例:

```
USE student_system;
```

说明: 类似 PowerShell 的 `cd` 命令, 进入某个数据库才能操作里面的表

删除数据库 (危险操作)

```
DROP DATABASE 数据库名;
```

 **警告:** 删除数据库会永久删除里面所有的表和数据, 无法恢复!

2. 表操作

查看所有表

```
SHOW TABLES;
```

说明: 显示当前数据库中所有的表

创建表

```
CREATE TABLE 表名 (  
    列名1 数据类型 约束,  
    列名2 数据类型 约束,
```

```
...  
);
```

示例:

```
CREATE TABLE students (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(50) NOT NULL,  
    age INT,  
    score FLOAT  
);
```

参数说明:

- **INT** : 整数类型
- **VARCHAR(50)** : 可变长度字符串, 最多50个字符
- **FLOAT** : 浮点数
- **PRIMARY KEY** : 主键, 唯一标识每一行
- **AUTO_INCREMENT** : 自动递增 (1, 2, 3...)
- **NOT NULL** : 该列不能为空

 **提示:** 最后一个字段后面不要加逗号, 直接跟右括号和分号

查看表结构

```
DESCRIBE 表名;  
-- 或简写  
DESC 表名;
```

示例:

```
DESC students;
```

删除表 (危险操作)

```
DROP TABLE 表名;
```

3. 数据操作（增删改查）

插入数据（INSERT）

插入单条数据：

```
INSERT INTO 表名 (列1, 列2, 列3) VALUES (值1, 值2, 值3);
```

示例：

```
INSERT INTO students (name, age, score) VALUES ('张三', 20, 85.5);
```

插入多条数据：

```
INSERT INTO 表名 (列1, 列2, 列3) VALUES  
(值1, 值2, 值3),  
(值4, 值5, 值6),  
(值7, 值8, 值9);
```

示例：

```
INSERT INTO students (name, age, score) VALUES  
( '李四', 19, 92.0),  
( '王五', 21, 78.5),  
( '赵六', 20, 88.0);
```

注意事项：

- 字符串用单引号 '张三'
- 数字直接写 20 或 85.5
- 必须用英文逗号 , , 不能用中文逗号 ，
- 如果列有 AUTO_INCREMENT , 插入时不用指定该列

查询数据 (SELECT)

查询所有数据:

```
SELECT * FROM 表名;
```

说明: * 表示所有列

查询特定列:

```
SELECT 列1, 列2 FROM 表名;
```

示例:

```
SELECT name, score FROM students;
```

条件查询 (WHERE) :

```
SELECT * FROM 表名 WHERE 条件;
```

示例:

```
-- 查询成绩大于85的学生
SELECT * FROM students WHERE score > 85;

-- 查询名字是张三的学生
SELECT * FROM students WHERE name = '张三';

-- 查询年龄在18到22岁之间的学生
SELECT * FROM students WHERE age >= 18 AND age <= 22;
```

排序 (ORDER BY) :

```
SELECT * FROM 表名 ORDER BY 列名 [ASC|DESC];
```

说明:

- **ASC** : 升序 (从小到大) , 默认
- **DESC** : 降序 (从大到小)

示例:

```
-- 按成绩从高到低排序
SELECT * FROM students ORDER BY score DESC;

-- 按年龄从小到大排序
SELECT * FROM students ORDER BY age ASC;
```

限制结果数量 (LIMIT) :

```
SELECT * FROM 表名 LIMIT 数量;
```

示例:

```
-- 只显示前3条记录
SELECT * FROM students LIMIT 3;
```

更新数据 (UPDATE)

```
UPDATE 表名 SET 列名 = 新值 WHERE 条件;
```

示例:

```
-- 把张三的成绩改成90
UPDATE students SET score = 90 WHERE name = '张三';

-- 把所有学生的年龄加1
UPDATE students SET age = age + 1;
```

 **危险警告:** 如果不加 **WHERE** 条件, 会修改所有行的数据!

```
-- 危险！会把所有学生的成绩都改成90  
UPDATE students SET score = 90;
```

删除数据 (DELETE)

```
DELETE FROM 表名 WHERE 条件;
```

示例:

```
-- 删除名字是王五的学生  
DELETE FROM students WHERE name = '王五';  
  
-- 删除成绩低于60的学生  
DELETE FROM students WHERE score < 60;
```

 **超级危险警告:** 如果不加 **WHERE** 条件, 会删除表中所有数据!

```
-- 危险！删除所有数据（表还在，数据全没）  
DELETE FROM students;
```

4. 统计函数

COUNT - 统计行数

```
SELECT COUNT(*) FROM 表名;
```

示例:

```
-- 统计学生总数  
SELECT COUNT(*) FROM students;
```



```
-- 统计成绩大于80的学生人数  
SELECT COUNT(*) FROM students WHERE score > 80;
```

AVG - 求平均值

```
SELECT AVG(列名) FROM 表名;
```

示例:

```
-- 求平均成绩  
SELECT AVG(score) FROM students;
```

MAX - 求最大值

```
SELECT MAX(列名) FROM 表名;
```

示例:

```
-- 求最高分  
SELECT MAX(score) FROM students;
```

MIN - 求最小值

```
SELECT MIN(列名) FROM 表名;
```

示例:

```
-- 求最低分  
SELECT MIN(score) FROM students;
```

SUM - 求总和

```
SELECT SUM(列名) FROM 表名;
```

示例:

```
-- 求总分
SELECT SUM(score) FROM students;
```

四、SQL 与 pandas 对比

操作	SQL 命令	pandas 写法
创建表	CREATE TABLE	pd.DataFrame()
查看所有数据	SELECT * FROM 表名	df
选择列	SELECT 列名 FROM 表名	df["列名"]
条件筛选	WHERE score > 85	df[df["score"] > 85]
排序	ORDER BY 列名 DESC	df.sort_values("列名", ascending=False)
更新数据	UPDATE SET 列=值 WHERE 条件	df.loc[条件, "列"] = 值
删除行	DELETE WHERE 条件	df = df[条件]
统计行数	COUNT(*)	len(df)
求平均值	AVG(列名)	df["列名"].mean()

五、常见错误与解决

1. 中文标点符号

错误: `VALUES('张三', 20)` 用了中文逗号

正确: `VALUES('张三', 20)` 必须用英文逗号

2. 最后一个字段多了逗号

错误:

```
CREATE TABLE books(  
    title VARCHAR(100),  
    price FLOAT,    ← 多了逗号  
);
```

正确:

```
CREATE TABLE books(  
    title VARCHAR(100),  
    price FLOAT    ← 没有逗号  
);
```

3. 忘记 WHERE 条件

危险:

```
UPDATE students SET score = 90;  -- 所有学生成绩都变90  
DELETE FROM students;           -- 删除所有数据
```

正确:

```
UPDATE students SET score = 90 WHERE name = '张三';  
DELETE FROM students WHERE score < 60;
```

4. 拼写错误

- `DELETE FORM` 错误 → 应该是 `DELETE FROM`
- `CREAT TABLE` 错误 → 应该是 `CREATE TABLE`
- `SELCT` 错误 → 应该是 `SELECT`

六、今日练习总结

练习任务：创建书店数据库 `bookstore` 和表 `books`

完成的操作：

1. 创建数据库和表（包含 id、书名、作者、价格、年份）
2. 插入5本书的数据
3. 查询价格超过35元的书
4. 按价格排序
5. 更新《活着》的价格
6. 删除1950年前出版的书
7. 计算剩余书籍的平均价格

结果验证：

- 删除2本书后剩余3本：《三体》《活着》《百年孤独》
- 平均价格： $(36 + 45 + 58) \div 3 = 46.33$ 元 

七、重要提醒

安全习惯（必须遵守）

-  `UPDATE` 和 `DELETE` 必须加 `WHERE` 条件
-  所有 SQL 语句必须以英文分号 `;` 结尾
-  字符串用单引号 `'张三'`，不是双引号
-  所有标点符号必须是英文（半角），不能用中文（全角）
-  主键设置 `AUTO_INCREMENT` 可以自动递增，插入时不用手动指定

下一步学习

Day 8 预告:

- 安装 `pymysql` 库
- 用 Python 代码连接 MySQL
- 在 .py 文件里执行 SQL 命令
- pandas 和 MySQL 互导数据

八、快速参考卡片

连接 MySQL

```
mysql -u root -p
```

数据库相关

SHOW DATABASES;	-- 查看所有数据库
CREATE DATABASE 数据库名;	-- 创建数据库
USE 数据库名;	-- 使用数据库
DROP DATABASE 数据库名;	-- 删除数据库

表相关

SHOW TABLES;	-- 查看所有表
DESC 表名;	-- 查看表结构
DROP TABLE 表名;	-- 删除表

增删改查

INSERT INTO 表名 (列) VALUES (值);	-- 插入
SELECT * FROM 表名 WHERE 条件;	-- 查询

```
UPDATE 表名 SET 列=值 WHERE 条件;    -- 更新
DELETE FROM 表名 WHERE 条件;         -- 删除
```

统计函数

```
SELECT COUNT(*) FROM 表名;    -- 统计行数
SELECT AVG(列名) FROM 表名;   -- 平均值
SELECT MAX(列名) FROM 表名;   -- 最大值
SELECT MIN(列名) FROM 表名;   -- 最小值
SELECT SUM(列名) FROM 表名;   -- 总和
```

学习日期: 2026-06-29 | Day 7 完成 

下一步: Day 8 - Python 连接 MySQL